

AUTOMATION

Automate actions with hooks

Run shell commands automatically when Claude Code edits files, finishes tasks, or needs input. Format code, send notifications, validate commands, and enforce project rules.

Hooks are user-defined shell commands that execute at specific points in Claude Code's lifecycle. They provide deterministic control over Claude Code's behavior, ensuring certain actions always happen rather than relying on the LLM to choose to run them. Use hooks to enforce project rules, automate repetitive tasks, and integrate Claude Code with your existing tools.

For decisions that require judgment rather than deterministic rules, you can also use [prompt-based hooks](#) or [agent-based hooks](#) that use a Claude model to evaluate conditions.

For other ways to extend Claude Code, see [skills](#) for giving Claude additional instructions and executable commands, [subagents](#) for running tasks in isolated contexts, and [plugins](#) for packaging extensions to share across projects.

 This guide covers common use cases and how to get started. For full event schemas, JSON input/output formats, and advanced features like async hooks and MCP tool hooks, see the [Hooks reference](#).

Set up your first hook

To create a hook, add a `hooks` block to a [settings file](#). This walkthrough creates a desktop notification hook, so you get alerted whenever Claude is waiting for your input instead of watching the terminal.

1 Add the hook to your settings

Open `~/.claude/settings.json` and add a `Notification` hook. The example below uses `osascript` for macOS; see [Get notified when Claude needs input](#) for Linux and Windows commands.

```

{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification
\"Claude Code needs your attention\" with title \"Claude
Code\"'"
          }
        ]
      }
    ]
  }
}

```

If your settings file already has a `hooks` key, add `Notification` as a sibling of the existing event keys rather than replacing the whole object. Each event name is a key inside the single `hooks` object:

```

{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [{ "type": "command", "command": "jq -r
'.tool_input.file_path' | xargs npx prettier --write" }]
      }
    ],
    "Notification": [
      {
        "matcher": "",
        "hooks": [{ "type": "command", "command": "osascript
-e 'display notification \"Claude Code needs your
attention\" with title \"Claude Code\"'" }]
      }
    ]
  }
}

```

You can also ask Claude to write the hook for you by describing what you want in the CLI.

2 Verify the configuration

Type `/hooks` to open the hooks browser. You'll see a list of all available hook events, with a count next to each event that has hooks configured. Select `Notification` to confirm your new hook appears in the list. Selecting the hook shows its details: the event, matcher, type, source file, and command.

3 Test the hook

Press `Esc` to return to the CLI. Ask Claude to do something that requires permission, then switch away from the terminal. You should receive a desktop notification.

 The `/hooks` menu is read-only. To add, modify, or remove hooks, edit your settings JSON directly or ask Claude to make the change.

What you can automate

Hooks let you run code at key points in Claude Code's lifecycle: format files after edits, block commands before they execute, send notifications when Claude needs input, inject context at session start, and more. For the full list of hook events, see the [Hooks reference](#).

Each example includes a ready-to-use configuration block that you add to a [settings file](#). The most common patterns:

- [Get notified when Claude needs input](#)
- [Auto-format code after edits](#)
- [Block edits to protected files](#)
- [Re-inject context after compaction](#)
- [Audit configuration changes](#)
- [Reload environment when directory or files change](#)
- [Auto-approve specific permission prompts](#)

For a production example of hooks that run a separate model review and feed findings back into the session, see [how the security-guidance plugin integrates with Claude Code](#).

Get notified when Claude needs input

Get a desktop notification whenever Claude finishes working and needs your input, so you can switch to other tasks without checking the terminal.

This hook uses the `Notification` event, which fires when Claude is waiting for input or permission. Each tab below uses the platform's native notification command. Add this to

`~/.claude/settings.json` :

macOS

```
{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification\n\"Claude Code needs your attention\" with title \"Claude Code\"'"
          }
        ]
      }
    ]
  }
}
```

► If no notification appears

Linux

```

{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "notify-send 'Claude Code' 'Claude Code
needs your attention'"
          }
        ]
      }
    ]
  }
}

```

Windows (PowerShell)

```

{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "powershell.exe -Command \"
[System.Reflection.Assembly]::LoadWithPartialName('System.Windows
.Forms'); [System.Windows.Forms.MessageBox]::Show('Claude Code
needs your attention', 'Claude Code')\""
          }
        ]
      }
    ]
  }
}

```

Match only `Bash` tool calls and log each command to a file. The `PostToolUse` event fires after the command completes, so `tool_input.command` contains what ran. The hook receives the event data as JSON on stdin, and `jq -r '.tool_input.command'` extracts just the command string, which `>>` appends to the log file:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.command' >>
~/ .claude/command-log.txt"
          }
        ]
      }
    ]
  }
}
```

MCP tools use a different naming convention than built-in tools: `mcp__<server>__<tool>`, where `<server>` is the MCP server name and `<tool>` is the tool it provides. For example, `mcp__github__search_repositories` or `mcp__filesystem__read_file`. Use a regex matcher to target all tools from a specific server, or match across servers with a pattern like `mcp__.*__write.*`. See **Match MCP tools** in the reference for the full list of examples. The command below extracts the tool name from the hook's JSON input with `jq` and writes it to stderr. Writing to stderr keeps stdout clean for JSON output and sends the message to the **debug log**:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "mcp__github__.*",
        "hooks": [
          {
            "type": "command",
            "command": "echo \"GitHub tool called: $(jq -r
'.tool_name')\" >&2"
          }
        ]
      }
    ]
  }
}
```

The `SessionEnd` event supports matchers on the reason the session ended. This hook only fires on `clear` (when you run `/clear`), not on normal exits:

```
{
  "hooks": {
    "SessionEnd": [
      {
        "matcher": "clear",
        "hooks": [
          {
            "type": "command",
            "command": "rm -f /tmp/claude-scratch-*.txt"
          }
        ]
      }
    ]
  }
}
```

The empty `matcher` fires on all notification types. To fire only on specific events, set it to one of these values:

Matcher	Fires when
<code>permission_prompt</code>	Claude needs you to approve a tool use
<code>idle_prompt</code>	Claude is done and waiting for your next prompt
<code>auth_success</code>	Authentication completes
<code>elicitation_dialog</code>	An MCP server opens an elicitation form
<code>elicitation_complete</code>	An MCP elicitation form is submitted or dismissed
<code>elicitation_response</code>	An MCP elicitation response is sent back to the server

Type `/hooks` and select `Notification` to confirm the hook is registered. For the full event schema, see the [Notification reference](#).

Auto-format code after edits

Automatically run [Prettier](#) on every file Claude edits, so formatting stays consistent without manual intervention.

This hook uses the `PostToolUse` event with an `Edit|Write` matcher, so it runs only after file-editing tools. On Claude Code v2.1.191 or later you can also write the matcher as `Edit,Write`, since `|` and `,` are interchangeable list separators for tool-name matchers on those versions. The command extracts the edited file path with `jq` and passes it to Prettier. Add this to `.claude/settings.json` in your project root:

```

{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.file_path' | xargs npx
prettier --write"
          }
        ]
      }
    ]
  }
}

```

❗ The Bash examples on this page use `jq` for JSON parsing. Install it with `brew install jq` (macOS), `apt-get install jq` (Debian/Ubuntu), or see [jq downloads](#).

Block edits to protected files

Prevent Claude from modifying sensitive files like `.env`, `package-lock.json`, or anything in `.git/`. Claude receives feedback explaining why the edit was blocked, so it can adjust its approach.

This example uses a separate script file that the hook calls. The script checks the target file path against a list of protected patterns and exits with code 2 to block the edit.

1 Create the hook script

Save this to `.claude/hooks/protect-files.sh` :

```
#!/bin/bash
# protect-files.sh

INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

PROTECTED_PATTERNS=".env" "package-lock.json" ".git/"

for pattern in "${PROTECTED_PATTERNS[@]}; do
    if [[ "$FILE_PATH" == *"$pattern"* ]]; then
        echo "Blocked: $FILE_PATH matches protected pattern
'$pattern'" >&2
        exit 2
    fi
done

exit 0
```

2 Make the script executable (macOS/Linux)

Hook scripts must be executable for Claude Code to run them:

```
chmod +x .claude/hooks/protect-files.sh
```

3 Register the hook

Add a `PreToolUse` hook to `.claude/settings.json` that runs the script before any `Edit` or `Write` tool call:

```

{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command":
"$\n$CLAUDE_PROJECT_DIR\n"/.claude/hooks/protect-files.sh"
          }
        ]
      }
    ]
  }
}

```

Re-inject context after compaction

When Claude's context window fills up, compaction summarizes the conversation to free space. This can lose important details. Use a `SessionStart` hook with a `compact` matcher to re-inject critical context after every compaction.

Any text your command writes to stdout is added to Claude's context. This example reminds Claude of project conventions and recent work. Add this to `.claude/settings.json` in your project root:

```

{
  "hooks": {
    "SessionStart": [
      {
        "matcher": "compact",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'Reminder: use Bun, not npm. Run bun
test before committing. Current sprint: auth refactor.'"
          }
        ]
      }
    ]
  }
}

```

You can replace the `echo` with any command that produces dynamic output, like `git log --online -5` to show recent commits. For injecting context on every session start, consider using **CLAUDE.md** instead. For environment variables, see **CLAUDE_ENV_FILE** in the reference.

Audit configuration changes

Track when settings or skills files change during a session. The `ConfigChange` event fires when an external process or editor modifies a configuration file, so you can log changes for compliance or block unauthorized modifications.

This example appends each change to an audit log. Add this to `~/.claude/settings.json` :

```

{
  "hooks": {
    "ConfigChange": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "jq -c '{timestamp: now | todate, source:
.source, file: .file_path}' >> ~/claude-config-audit.log"
          }
        ]
      }
    ]
  }
}

```

The matcher filters by configuration type: `user_settings` , `project_settings` , `local_settings` , `policy_settings` , or `skills` . To block a change from taking effect, exit with code 2 or return `{"decision": "block"}` . See the [ConfigChange reference](#) for the full input schema.

Reload environment when directory or files change

Some projects set different environment variables depending on which directory you are in. Tools like [direnv](#) do this automatically in your shell, but Claude's Bash tool does not pick up those changes on its own.

Pairing a `SessionStart` hook with a `CwdChanged` hook fixes this. `SessionStart` loads the variables for the directory you launch in, and `CwdChanged` reloads them each time Claude changes directory. Both write to `CLAUDE_ENV_FILE` , which Claude Code runs as a script preamble before each Bash command. Add this to `~/ .claude/settings.json` :

```

{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "direnv export bash >
\"$CLAUDE_ENV_FILE\"
          }
        ]
      }
    ],
    "CwdChanged": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "direnv export bash >
\"$CLAUDE_ENV_FILE\"
          }
        ]
      }
    ]
  }
}

```

Run `direnv allow` once in each directory that has an `.envrc` so `direnv` is permitted to load it. If you use `devbox` or `nix` instead of `direnv`, the same pattern works with `devbox shellenv` or `devbox global shellenv` in place of `direnv export bash`.

To react to specific files instead of every directory change, use `FileChanged` with a `matcher` listing the filenames to watch, separated by `|`. To build the watch list, this value is split into literal filenames rather than evaluated as a regex. See [FileChanged](#) for how the same value also filters which hook groups run when a file changes. This example watches `.envrc` and `.env` in the working directory:

```

{
  "hooks": {
    "FileChanged": [
      {
        "matcher": ".envrc|.env",
        "hooks": [
          {
            "type": "command",
            "command": "direnv export bash >
\"$CLAUDE_ENV_FILE\"
          }
        ]
      }
    ]
  }
}

```

See the [CwdChanged](#) and [FileChanged](#) reference entries for input schemas, `watchPaths` output, and `CLAUDE_ENV_FILE` details.

Auto-approve specific permission prompts

Skip the approval dialog for tool calls you always allow. This example auto-approves `ExitPlanMode`, the tool Claude calls when it finishes presenting a plan and asks to proceed, so you aren't prompted every time a plan is ready.

Unlike the exit-code examples above, auto-approval requires your hook to write a JSON decision to stdout. A `PermissionRequest` hook fires when Claude Code is about to show a permission dialog, and returning `"behavior": "allow"` answers it on your behalf.

The matcher scopes the hook to `ExitPlanMode` only, so no other prompts are affected. Add this to `~/.claude/settings.json`:

```

{
  "hooks": {
    "PermissionRequest": [
      {
        "matcher": "ExitPlanMode",
        "hooks": [
          {
            "type": "command",
            "command": "echo '{\"hookSpecificOutput\":
{\"hookEventName\": \"PermissionRequest\", \"decision\":
{\"behavior\": \"allow\"}}}'"
          }
        ]
      }
    ]
  }
}

```

When the hook approves, Claude Code exits plan mode and restores whatever permission mode was active before you entered plan mode. The transcript shows “Allowed by PermissionRequest hook” where the dialog would have appeared. The hook path always keeps the current conversation: it cannot clear context and start a fresh implementation session the way the dialog can.

To set a specific permission mode instead, your hook’s output can include an `updatedPermissions` array with a `setMode` entry. The `mode` value is any permission mode like `default` , `acceptEdits` , or `bypassPermissions` , and `destination: "session"` applies it for the current session only.

❗ `bypassPermissions` only applies if the session was launched with bypass mode already available: `--dangerously-skip-permissions` , `--permission-mode bypassPermissions` , `--allow-dangerously-skip-permissions` , or `permissions.defaultMode: "bypassPermissions"` in settings, and not disabled by `permissions.disableBypassPermissionsMode` . It is never persisted as `defaultMode` .

To switch the session to `acceptEdits` , your hook writes this JSON to stdout:

```

{
  "hookSpecificOutput": {
    "hookEventName": "PermissionRequest",
    "decision": {
      "behavior": "allow",
      "updatedPermissions": [
        { "type": "setMode", "mode": "acceptEdits",
"destination": "session" }
      ]
    }
  }
}

```

Keep the matcher as narrow as possible. Matching on `.*` or leaving the matcher empty would auto-approve every permission prompt, including file writes and shell commands. See the [PermissionRequest reference](#) for the full set of decision fields.

How hooks work

Hook events fire at specific lifecycle points in Claude Code. When an event fires, all matching hooks run in parallel, and identical hook commands are automatically deduplicated. The table below shows each event and when it triggers:

Event	When it fires
SessionStart	When a session begins or resumes
Setup	When you start Claude Code with <code>--init-only</code> , or with <code>--init</code> or <code>--maintenance</code> in <code>-p</code> mode. For one-time preparation in CI or scripts
UserPromptSubmit	When you submit a prompt, before Claude processes it
UserPromptExpansion	When a user-typed command expands into a prompt, before it reaches Claude. Can block the expansion
PreToolUse	Before a tool call executes. Can block it
PermissionRequest	When a permission dialog appears
PermissionDenied	When a tool call is denied by the auto mode classifier. Return <code>{retry: true}</code> to tell the model it may retry the denied tool call
PostToolUse	After a tool call succeeds
PostToolUseFailure	After a tool call fails

Event	When it fires
PostToolBatch	After a full batch of parallel tool calls resolves, before the next model call
Notification	When Claude Code sends a notification
MessageDisplay	While assistant message text is displayed
SubagentStart	When a subagent is spawned
SubagentStop	When a subagent finishes
TaskCreated	When a task is being created via TaskCreate
TaskCompleted	When a task is being marked as completed
Stop	When Claude finishes responding
StopFailure	When the turn ends due to an API error. Output and exit code are ignored
TeammateIdle	When an agent team teammate is about to go idle
InstructionsLoaded	When a CLAUDE.md or .claude/rules/*.md file is loaded into context. Fires at session start and when files are lazily loaded during a session
ConfigChange	When a configuration file changes during a session
CwdChanged	When the working directory changes, for example when Claude executes a cd command. Useful for reactive environment management with tools like direnv
FileChanged	When a watched file changes on disk. The matcher field specifies which filenames to watch
WorktreeCreate	When a worktree is being created via --worktree or isolation: "worktree" . Replaces default git behavior
WorktreeRemove	When a worktree is being removed, either at session exit or when a subagent finishes
PreCompact	Before context compaction
PostCompact	After context compaction completes
Elicitation	When an MCP server requests user input during a tool call
ElicitationResult	After a user responds to an MCP elicitation, before the response is sent back to the server
SessionEnd	When a session terminates

Each hook has a type that determines how it runs. Most hooks use "type": "command" , which runs a shell command. Four other types are available:

- "type": "http" : POST event data to a URL. See HTTP hooks.
- "type": "mcp_tool" : call a tool on an already-connected MCP server. See MCP tool hooks.

- `"type": "prompt"` : single-turn LLM evaluation. See [Prompt-based hooks](#).
- `"type": "agent"` : multi-turn verification with tool access. Agent hooks are experimental and may change. See [Agent-based hooks](#).

Combine results from multiple hooks

When multiple hooks match the same event, every hook's command runs to completion before Claude Code merges the results. One hook returning `deny` does not stop sibling hooks from executing. Don't rely on one hook's `deny` to suppress side effects in another hook.

After all matching hooks finish, Claude Code combines their outputs. For `PreToolUse` permission decisions, the most restrictive answer wins, in the order `deny` , `defer` , `ask` , `allow` . Text from `additionalContext` is kept from every hook and passed to Claude together.

The example below registers two `PreToolUse` hooks on `Bash` . The first appends every command to a log file and exits 0. The second runs a script that exits 2 to deny when the command contains `rm` `-rf` :

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r .tool_input.command >>
~/\.claude/bash.log"
          },
          {
            "type": "command",
            "command":
"\\"$CLAUDE_PROJECT_DIR\\"/.claude/hooks/block-rm-rf.sh"
          }
        ]
      }
    ]
  }
}
```

When Claude tries to run `rm -rf /tmp/build` , both hooks execute in parallel. The logging hook

writes the command to `~/.claude/bash.log` and exits 0, which reports no decision. The guardrail hook exits 2, which denies the tool call. The deny wins, so Claude Code blocks the command and shows Claude the guardrail's stderr. The log entry is still written because the logging hook already ran.

Read input and return output

Hooks communicate with Claude Code through stdin, stdout, stderr, and exit codes. When an event fires, Claude Code passes event-specific data as JSON to your script's stdin. Your script reads that data, does its work, and tells Claude Code what to do next via the exit code.

Hook input

Every event includes common fields like `session_id` and `cwd`, but each event type adds different data. For example, when Claude runs a Bash command, a `PreToolUse` hook receives something like this on stdin:

```
{
  "session_id": "abc123",           // unique ID for this session
  "cwd": "/Users/sarah/myproject", // working directory when the
event fired
  "hook_event_name": "PreToolUse", // which event triggered this
hook
  "tool_name": "Bash",             // the tool Claude is about to
use
  "tool_input": {                  // the arguments Claude passed
to the tool
    "command": "npm test"          // for Bash, this is the shell
command
  }
}
```

Your script can parse that JSON and act on any of those fields. `UserPromptSubmit` hooks get the `prompt` text instead, `SessionStart` hooks get the `source` (startup, resume, clear, compact), and so on. See [Common input fields](#) in the reference for shared fields, and each event's section for event-specific schemas.

Hook output

Your script tells Claude Code what to do next by writing to stdout or stderr and exiting with a

specific code. For example, a `PreToolUse` hook that wants to block a command:

```
#!/bin/bash
INPUT=$(cat)
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command')

if echo "$COMMAND" | grep -q "drop table"; then
    echo "Blocked: dropping tables is not allowed" >&2 # stderr
    # becomes Claude's feedback
    exit 2 # exit 2 =
    # block the action
fi

exit 0 # exit 0 = no decision; the normal permission flow
# applies
```

The exit code determines what happens next:

- **Exit 0:** the hook reports no objection and the action proceeds normally. For a `PreToolUse` hook this doesn't approve the tool call: the normal permission flow still applies. For `UserPromptSubmit`, `UserPromptExpansion`, and `SessionStart` hooks, anything you write to stdout is added to Claude's context.
- **Exit 2:** the action is blocked. Write a reason to stderr, and Claude receives it as feedback so it can adjust. Some events cannot be blocked: for `SessionStart`, `Setup`, `Notification`, and others, exit 2 shows stderr to the user and execution continues. See exit code 2 behavior per event for the full list.
- **Any other exit code:** the action proceeds. The transcript shows a `<hook name> hook error` notice followed by the first line of stderr; the full stderr goes to the debug log.

Structured JSON output

Exit codes only let you block or stay silent. For more control, exit 0 and print a JSON object to stdout instead.

❗ Use exit 2 to block with a stderr message, or exit 0 with JSON for structured control. Don't mix them: Claude Code ignores JSON when you exit 2.

For example, a `PreToolUse` hook can deny a tool call and tell Claude why, or escalate it to the user for approval:

```
{
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Use rg instead of grep for
better performance"
  }
}
```

With `"deny"`, Claude Code cancels the tool call and feeds `permissionDecisionReason` back to Claude. These `permissionDecision` values are specific to `PreToolUse`:

- `"allow"`: skip the interactive permission prompt. Deny and ask rules, including enterprise managed deny lists, still apply
- `"deny"`: cancel the tool call and send the reason to Claude
- `"ask"`: show the permission prompt to the user as normal

A fourth value, `"defer"`, is available in **non-interactive mode** with the `-p` flag. It exits the process with the tool call preserved so an Agent SDK wrapper can collect input and resume. See **Defer a tool call for later** in the reference.

Returning `"allow"` skips the interactive prompt but does not override **permission rules**. If a deny rule matches the tool call, the call is blocked even when your hook returns `"allow"`. If an ask rule matches, the user is still prompted. This means deny rules from any settings scope, including **managed settings**, always take precedence over hook approvals.

Other events use different decision patterns. For example, `PostToolUse` and `Stop` hooks use a top-level `decision: "block"` field, while `PermissionRequest` uses `hookSpecificOutput.decision.behavior`. See the **summary table** in the reference for a full breakdown by event.

For `UserPromptSubmit` hooks, use `additionalContext` instead to inject text into Claude's context. Prompt-based hooks (`type: "prompt"`) handle output differently: see **Prompt-based hooks**.

Filter hooks with matchers

Without a matcher, a hook fires on every occurrence of its event. Matchers let you narrow that down. For example, if you want to run a formatter only after file edits (not after every tool call), add a matcher to your `PostToolUse` hook:

```

{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "prettier --write ..."
        ]
      }
    ]
  }
}

```

The `"Edit|Write"` matcher fires only when Claude uses the `Edit` or `Write` tool, not when it uses `Bash`, `Read`, or any other tool. See [Matcher patterns](#) for how plain names and regular expressions are evaluated.

❗ Claude can also create or modify files by running shell commands through the `Bash` tool. If your hook must see every file change, such as for compliance scanning or audit logging, add a `Stop` hook that scans the working tree once per turn. For per-call coverage instead, also match `Bash` and have your script list modified and untracked files with `git status --porcelain`.

Each event type matches on a specific field:

Event	What the matcher filters	Example matcher values
PreToolUse , PostToolUse , PostToolUseFailure , PermissionRequest , PermissionDenied	tool name	Bash , Edit Write , mcp__.*
SessionStart	how the session started	startup , resume , clear , compact
Setup	which CLI flag triggered setup	init , maintenance
SessionEnd	why the session ended	clear , resume , logout , prompt_input_exit , bypass_permissions_disabled , other
Notification	notification type	permission_prompt , idle_prompt , auth_success , elicitation_dialog , elicitation_complete ,

Event	What the matcher filters	Example matcher values
		elicitation_response
SubagentStart	agent type	general-purpose , Explore , Plan , Or custom agent names
PreCompact , PostCompact	what triggered compaction	manual , auto
SubagentStop	agent type	same values as SubagentStart
ConfigChange	configuration source	user_settings , project_settings , local_settings , policy_settings , skills
StopFailure	error type	rate_limit , overloaded , authentication_failed , oauth_org_not_allowed , billing_error , invalid_request , model_not_found , server_error , max_output_tokens , unknown
InstructionsLoaded	load reason	session_start , nested_traversal , path_glob_match , include , compact
Elicitation	MCP server name	your configured MCP server names
ElicitationResult	MCP server name	same values as Elicitation
FileChanged	literal filenames to watch (see <u>FileChanged</u>)	.envrc .env
UserPromptExpansion	command name	your skill or command names
UserPromptSubmit , PostToolBatch , Stop , TeammateIdle , TaskCreated , TaskCompleted , WorktreeCreate , WorktreeRemove , CwdChanged , MessageDisplay	no matcher support	always fires on every occurrence

A few more examples showing matchers on different event types:

Log every Bash command

--

```
{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification
\"Claude Code needs your attention\" with title \"Claude Code\"'"
          }
        ]
      }
    ]
  }
}
```

► If no notification appears

Match MCP tools

```

{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "notify-send 'Claude Code' 'Claude Code
needs your attention'"
          }
        ]
      }
    ]
  }
}

```

Clean up on session end

```

{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "powershell.exe -Command \"
[System.Reflection.Assembly]::LoadWithPartialName('System.Windows
.Forms'); [System.Windows.Forms.MessageBox]::Show('Claude Code
needs your attention', 'Claude Code')\""
          }
        ]
      }
    ]
  }
}

```

Match only `Bash` tool calls and log each command to a file. The `PostToolUse` event fires after the command completes, so `tool_input.command` contains what ran. The hook receives the event data as JSON on stdin, and `jq -r '.tool_input.command'` extracts just the command string, which `>>` appends to the log file:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.command' >>
~/claude/command-log.txt"
          }
        ]
      }
    ]
  }
}
```

MCP tools use a different naming convention than built-in tools: `mcp__<server>__<tool>`, where `<server>` is the MCP server name and `<tool>` is the tool it provides. For example, `mcp__github__search_repositories` or `mcp__filesystem__read_file`. Use a regex matcher to target all tools from a specific server, or match across servers with a pattern like `mcp__.*__write.*`. See **Match MCP tools** in the reference for the full list of examples. The command below extracts the tool name from the hook's JSON input with `jq` and writes it to stderr. Writing to stderr keeps stdout clean for JSON output and sends the message to the **debug log**:

```

{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "mcp__github__.*",
        "hooks": [
          {
            "type": "command",
            "command": "echo \"GitHub tool called: $(jq -r
'.tool_name')\" >&2"
          }
        ]
      }
    ]
  }
}

```

The `SessionEnd` event supports matchers on the reason the session ended. This hook only fires on `clear` (when you run `/clear`), not on normal exits:

```

{
  "hooks": {
    "SessionEnd": [
      {
        "matcher": "clear",
        "hooks": [
          {
            "type": "command",
            "command": "rm -f /tmp/claude-scratch-*.txt"
          }
        ]
      }
    ]
  }
}

```

For full matcher syntax, see the [Hooks reference](#).

Filter by tool name and arguments with the `if` field

ⓘ The `if` field requires Claude Code v2.1.85 or later. Earlier versions ignore it and run the hook on every matched call.

The `if` field uses **permission rule syntax** to filter hooks by tool name and arguments together, so the hook process only spawns when the tool call matches. This goes beyond `matcher`, which filters at the group level by tool name only.

For example, to run a hook only when Claude uses `git` commands rather than all Bash commands:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "if": "Bash(git *)",
            "command":
"$\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/check-git-policy.sh"
          }
        ]
      }
    ]
  }
}
```

Whether your hook command runs depends on the shape of your `if` pattern and the Bash command Claude is invoking:

if pattern	Bash command	Hook runs?	Why
Bash(git *)	git push	yes	command name matches
Bash(git *)	npm test && git push	yes	each subcommand is checked; git push matches

if pattern	Bash command	Hook runs?	Why
Bash(git *)	echo \$(git log)	yes	commands inside <code>\$()</code> and backticks are checked; <code>git log</code> matches
Bash(git *)	echo \$(date)	no	no subcommand matches <code>git *</code>
Bash(git push *)	echo \$(date)	yes	patterns that specify more than the command name run the hook anyway on <code>\$()</code> , backticks, or <code>\$VAR</code>

The filter also fails open, running your hook regardless of pattern, when the Bash command cannot be parsed. Because the filter is best-effort, use the permission system rather than a hook to enforce a hard allow or deny.

The `if` field accepts the same patterns as permission rules: `"Bash(git *)"` , `"Edit(*.ts)"` , and so on. To match multiple tool names, use separate handlers each with its own `if` value, or match at the `matcher` level where pipe alternation is supported.

`if` only works on tool events: `PreToolUse` , `PostToolUse` , `PostToolUseFailure` , `PermissionRequest` , and `PermissionDenied` . Adding it to any other event prevents the hook from running.

Configure hook location

Where you add a hook determines its scope:

Location	Scope	Shareable
<code>~/.claude/settings.json</code>	All your projects	No, local to your machine
<code>.claude/settings.json</code>	Single project	Yes, can be committed to the repo
<code>.claude/settings.local.json</code>	Single project	No, gitignored when Claude Code creates it
Managed policy settings	Organization-wide	Yes, admin-controlled
<u>Plugin</u> <code>hooks/hooks.json</code>	When plugin is enabled	Yes, bundled with the plugin
<u>Skill</u> or <u>agent</u> frontmatter	While the skill or agent is active	Yes, defined in the component file

Run `/hooks` in Claude Code to browse all configured hooks grouped by event. To disable hooks, set `"disableAllHooks": true` in your settings file. Hooks configured in managed settings still run unless `disableAllHooks` is also set there.

If you edit settings files directly while Claude Code is running, the file watcher normally picks up hook changes automatically.

Prompt-based hooks

For decisions that require judgment rather than deterministic rules, use `type: "prompt"` hooks. Instead of running a shell command, Claude Code sends your prompt and the hook's input data to a Claude model (Haiku by default) to make the decision. You can specify a different model with the `model` field if you need more capability.

The model's only job is to return a yes/no decision as JSON:

- `"ok": true` : the action proceeds
- `"ok": false` : what happens depends on the event:
 - `Stop` and `SubagentStop` : the `reason` is fed back to Claude so it keeps working
 - `PreToolUse` : the tool call is denied and the `reason` is returned to Claude as the tool error, so it can adjust and continue
 - `PostToolUse` , `PostToolBatch` , `UserPromptSubmit` , and `UserPromptExpansion` : the turn ends and the `reason` appears in the chat as a warning line

This example uses a `Stop` hook to ask the model whether all requested tasks are complete. If the model returns `"ok": false` , Claude keeps working and uses the `reason` as its next instruction:

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "prompt",
            "prompt": "Check if all tasks are complete. If not,
respond with {\"ok\": false, \"reason\": \"what remains to be
done\"}."
          }
        ]
      }
    ]
  }
}
```

For full configuration options, see [Prompt-based hooks](#) in the reference.

Agent-based hooks

⚠ Agent hooks are experimental. Behavior and configuration may change in future releases. For production workflows, prefer [command hooks](#).

When verification requires inspecting files or running commands, use `type: "agent"` hooks. Unlike prompt hooks which make a single LLM call, agent hooks spawn a subagent that can read files, search code, and use other tools to verify conditions before returning a decision.

Agent hooks use the same `"ok"` / `"reason"` response format as prompt hooks, but with a longer default timeout of 60 seconds and up to 50 tool-use turns.

This example verifies that tests pass before allowing Claude to stop:

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "agent",
            "prompt": "Verify that all unit tests pass. Run the
test suite and check the results. $ARGUMENTS",
            "timeout": 120
          }
        ]
      }
    ]
  }
}
```

Use prompt hooks when the hook input data alone is enough to make a decision. Use agent hooks when you need to verify something against the actual state of the codebase.

For full configuration options, see [Agent-based hooks](#) in the reference.

HTTP hooks

Use `type: "http"` hooks to POST event data to an HTTP endpoint instead of running a shell command. The endpoint receives the same JSON that a command hook would receive on stdin, and returns results through the HTTP response body using the same JSON format.

HTTP hooks are useful when you want a web server, cloud function, or external service to handle hook logic: for example, a shared audit service that logs tool use events across a team.

This example posts every tool use to a local logging service:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "hooks": [
          {
            "type": "http",
            "url": "http://localhost:8080/hooks/tool-use",
            "headers": {
              "Authorization": "Bearer $MY_TOKEN"
            },
            "allowedEnvVars": ["MY_TOKEN"]
          }
        ]
      }
    ]
  }
}
```

The endpoint should return a JSON response body using the same **output format** as command hooks. To block a tool call, return a 2xx response with the appropriate `hookSpecificOutput` fields. HTTP status codes alone cannot block actions.

Header values support environment variable interpolation using `$VAR_NAME` or `${VAR_NAME}` syntax. Only variables listed in the `allowedEnvVars` array are resolved; all other `$VAR` references remain empty.

For full configuration options and response handling, see **HTTP hooks** in the reference.

Limitations and troubleshooting

Limitations

- Command hooks communicate through stdout, stderr, and exit codes only. They cannot trigger `/` commands or tool calls. Text returned via `additionalContext` is injected as a system reminder that Claude reads as plain text. HTTP hooks communicate through the response body instead.
- Hook timeouts vary by type. Override per hook with the `timeout` field in seconds.
 - `command` , `http` , `mcp_tool` : 10 minutes. `UserPromptSubmit` lowers these to 30 seconds, and `MessageDisplay` lowers them to 10 seconds.
 - `prompt` : 30 seconds.
 - `agent` : 60 seconds.
- `PostToolUse` hooks cannot undo actions since the tool has already executed.
- `PermissionRequest` hooks do not fire in **non-interactive mode** (`-p`). Use `PreToolUse` hooks for automated permission decisions.
- `Stop` hooks fire whenever Claude finishes responding, not only at task completion. They do not fire on user interrupts. API errors fire **StopFailure** instead.
- When multiple `PreToolUse` hooks return **updatedInput** to rewrite a tool's arguments, the last one to finish wins. Since hooks run in parallel, the order is non-deterministic. Avoid having more than one hook modify the same tool's input.

Hooks and permission modes

`PreToolUse` hooks fire before any permission-mode check. A hook that returns

`permissionDecision: "deny"` blocks the tool even in `bypassPermissions` mode or with `--dangerously-skip-permissions` . This lets you enforce policy that users cannot bypass by changing their permission mode.

The reverse is not true: a hook returning `"allow"` does not bypass deny rules from settings. Hooks can tighten restrictions but not loosen them past what permission rules allow.

Hook not firing

The hook is configured but never executes.

- Run `/hooks` and confirm the hook appears under the correct event

- Check that the matcher pattern matches the tool name exactly (matchers are case-sensitive)
- Verify you're triggering the right event type (e.g., `PreToolUse` fires before tool execution, `PostToolUse` fires after)
- If using `PermissionRequest` hooks in non-interactive mode (`-p`), switch to `PreToolUse` instead

Hook error in output

You see a message like “PreToolUse hook error: ...” in the transcript.

- Your script exited with a non-zero code unexpectedly. Test it manually by piping sample JSON:

```
echo '{"tool_name":"Bash","tool_input":{"command":"ls"}}' |
./my-hook.sh
echo $? # Check the exit code
```

- If you see “command not found”, use absolute paths or `${CLAUDE_PROJECT_DIR}` to reference scripts. To avoid shell quoting entirely, add `"args": []` to switch to **exec form**, which spawns the script directly without a shell
- If you see “jq: command not found”, install `jq` or use Python/Node.js for JSON parsing
- If the script isn't running at all, make it executable: `chmod +x ./my-hook.sh`

`/hooks` shows no hooks configured

You edited a settings file but the hooks don't appear in the menu.

- File edits are normally picked up automatically. If they haven't appeared after a few seconds, the file watcher may have missed the change: restart your session to force a reload.
- Verify your JSON is valid (trailing commas and comments are not allowed)
- Confirm the settings file is in the correct location: `.claude/settings.json` for project hooks, `~/.claude/settings.json` for global hooks

Stop hook hits the block cap

Claude keeps working instead of stopping, then ends the turn with a warning that the Stop hook blocked too many consecutive times.

Claude Code overrides a Stop hook after it blocks 8 times in a row without progress. Your hook script needs to check whether it already triggered a continuation. Parse the `stop_hook_active` field from the JSON input and exit early if it's `true` :

```
#!/bin/bash
INPUT=$(cat)
if [ "$(echo "$INPUT" | jq -r '.stop_hook_active')" = "true" ];
then
    exit 0 # Allow Claude to stop
fi
# ... rest of your hook logic
```

If your hook legitimately needs more than eight iterations to converge, raise the cap with

`CLAUDE_CODE_STOP_HOOK_BLOCK_CAP` .

JSON validation failed

Claude Code shows a JSON parsing error even though your hook script outputs valid JSON.

When Claude Code runs a shell-form command hook (one without `args`), it spawns `sh -c` on macOS and Linux or Git Bash on Windows by default. This shell is non-interactive, but Git Bash and some configurations (such as `BASH_ENV` pointing at `~/.bashrc`) still source your profile. If that profile contains unconditional `echo` statements, the output gets prepended to your hook's JSON:

```
Shell ready on arm64
{"decision": "block", "reason": "Not allowed"}
```

Claude Code tries to parse this as JSON and fails. To fix this, wrap echo statements in your shell profile so they only run in interactive shells:

```
# In ~/.zshrc or ~/.bashrc
if [[ $- == *i* ]]; then
    echo "Shell ready"
fi
```

The `$-` variable contains shell flags, and `i` means interactive. Hooks run in non-interactive shells, so the echo is skipped.

Debug techniques

The transcript view, toggled with `Ctrl+0`, shows a one-line summary for each hook that fired: success is silent, blocking errors show stderr, and non-blocking errors show a `<hook name> hook error` notice followed by the first line of stderr.

For full execution details including which hooks matched, their exit codes, stdout, and stderr, read the debug log. Start Claude Code with `claude --debug-file /tmp/claude.log` to write to a known path, then `tail -f /tmp/claude.log` in another terminal. If you started without that flag, run `/debug` mid-session to enable logging and find the log path.

Learn more

- **Hooks reference**: full event schemas, JSON output format, async hooks, and MCP tool hooks
- **Security considerations**: review before deploying hooks in shared or production environments
- **Bash command validator example**: complete reference implementation